DERYA POLAT

LAB 4

PRELİMİNARY

03.12.2025

FSM Type and Explanation

I will use Moore type FSM.

Timing Behavior

I chose Moore FSM because outputs do not change when inputs change, they change only when on clock edges. This provides the controller with predictable timing (It is important when assigning cars to zones or starting timers.). It is better if the outputs change only at the clock edge since the controller only lets one car in each clock cycle. This makes the system's behavior easier to understand and prevents random output changes when the inputs change suddenly.

Output Stability

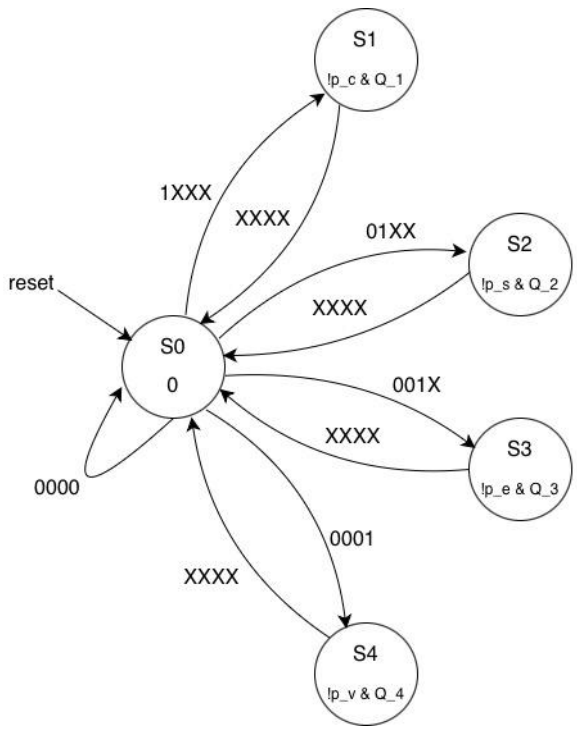
In Moore FSM, outputs depend on current state, not directly on inputs. This makes the output signals more stable. With a Moore machine, the outputs remain steady throughout the entire clock cycle.

Implementation Complexity

Moore FSM is simpler to implement in this project because each output corresponds to a state and this makes the logic easier to organize and reduces the chance of mid-cycle output changes.

FSM Transition Diagram, State Encodings, Next State, Output Tables and Boolean Equations, FSM Schematic

FSM Transition Diagram:



State Encodings

State	Binary Encoding
S0	000
S1	001
S2	010
S3	011
S4	100

Next State Table:

Current State			Input								Next Sate		
S_2	S_1	S_0	q_c	q_s	q_e	q_v	p_c	p_s	p_e	p_v	S'_2	S'_1	S'_0
0	0	0	1	X	X	X	X	X	X	X	0	0	1
0	0	0	0	1	X	X	X	X	X	X	0	1	0
0	0	0	0	0	1	X	X	X	X	X	0	1	1
0	0	0	0	0	0	1	X	X	X	X	1	0	0
0	0	0	0	0	0	0	X	X	X	X	0	0	0
0	0	1	X	X	X	X	X	X	X	X	0	0	0
0	1	0	X	X	X	X	X	X	X	X	0	0	0
0	1	1	X	X	X	X	X	X	X	X	0	0	0
1	0	0	X	X	X	X	X	X	X	X	0	0	0

Output Table

Current State			Input (8-bit bus)								Output (5-bit bus)				
S_2	S_1	S_0	q_c	q_s	q_e	q_v	p_c	p_s	p_e	p_v	r_c	r_s	r_e	r_v	r_q
0	0	0	X	X	X	X	X	X	X	X	0	0	0	0	0
0	0	1	X	X	X	X	0	X	X	X	1	0	0	0	0
0	0	1	X	X	X	X	1	X	X	X	0	0	0	0	1
0	1	0	X	X	X	X	X	0	X	X	0	1	0	0	0
0	1	0	X	X	X	X	X	1	X	X	0	0	0	0	1
0	1	1	X	X	X	X	X	X	0	X	0	0	1	0	0
0	1	1	X	X	X	X	X	X	1	X	0	0	0	0	1
1	0	0	X	X	X	X	X	X	X	0	0	0	0	1	0
1	0	0	X	X	X	X	X	X	X	1	0	0	0	0	1

Next-State and Output Boolean Equations:

$$S'_2 = (\overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0}) \cdot (\overline{q_c} \cdot \overline{q_s} \cdot \overline{q_e} \cdot q_v)$$

$$S'_1 = (\overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0}) \cdot (\overline{q_c} \cdot (q_s + q_e))$$

$$S'_0 = (\overline{S_2} \cdot \overline{S_1} \cdot \overline{S_0}) \cdot (q_c + (\overline{q_s} \cdot q_e))$$

$$r_c = (\overline{S_2} \cdot \overline{S_1} \cdot S_0) \cdot \overline{p_c}$$

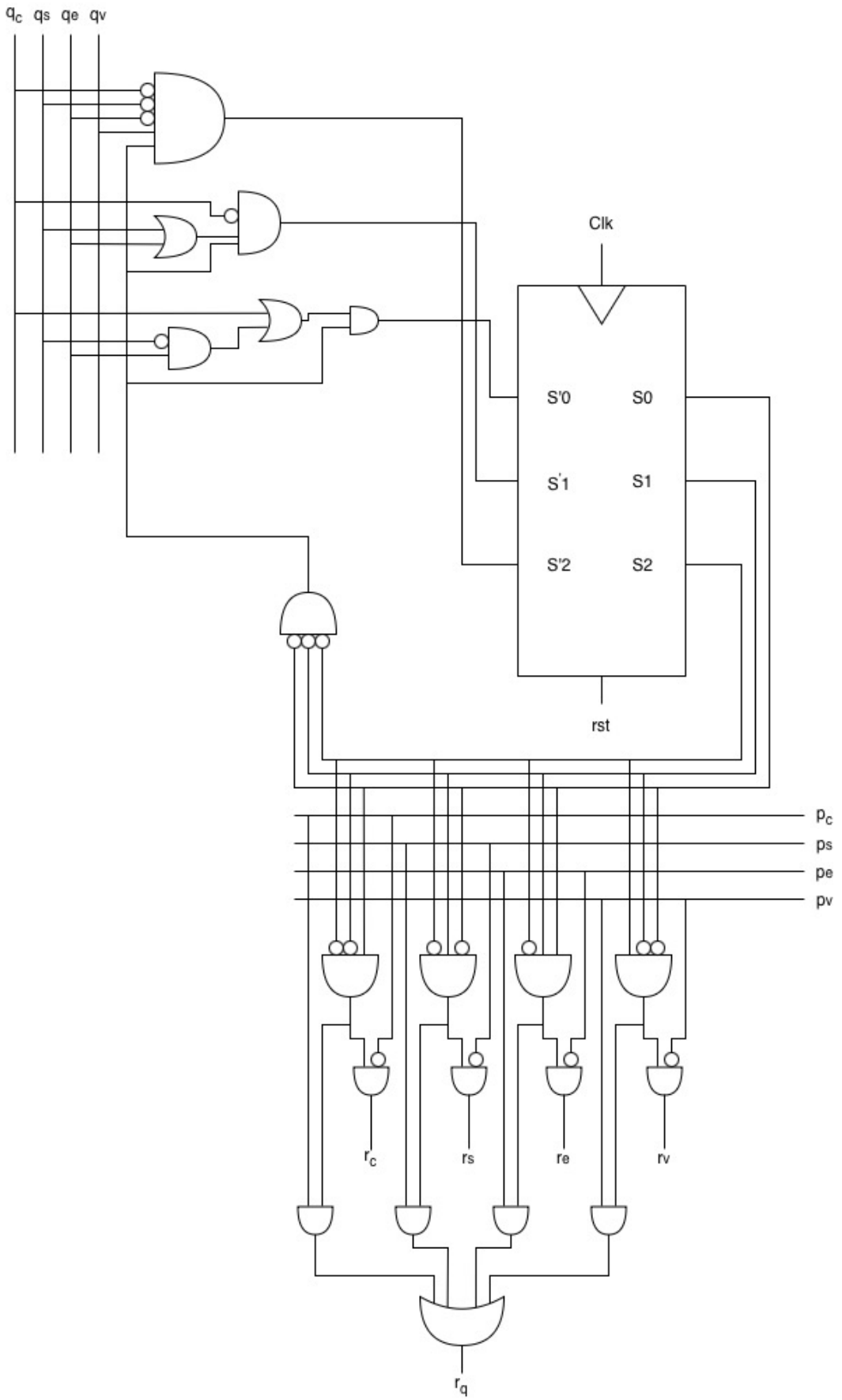
$$r_s = (\overline{S_2} \cdot S_1 \cdot \overline{S_0}) \cdot \overline{p_s}$$

$$r_e = (\overline{S_2} \cdot S_1 \cdot S_0) \cdot \overline{p_e}$$

$$r_v = (S_2 \cdot \overline{S_1} \cdot \overline{S_0}) \cdot \overline{p_v}$$

$$r_q = ((\overline{S_2} \cdot \overline{S_1} \cdot S_0) \cdot p_c) + ((\overline{S_2} \cdot S_1 \cdot \overline{S_0}) \cdot p_s) + ((\overline{S_2} \cdot S_1 \cdot S_0) \cdot p_e) + ((S_2 \cdot \overline{S_1} \cdot \overline{S_0}) \cdot p_v)$$

FSM Schematic:



8-to-1 Decoder

$$Q_1 = (\overline{S_2} \cdot \overline{S_1} \cdot S_0)$$

$$Q_2 = (\overline{S_2} \cdot S_1 \cdot \overline{S_0})$$

$$Q_3 = (\overline{S_2} \cdot S_1 \cdot S_0)$$

$$Q_4 = (S_2 \cdot \overline{S_1} \cdot \overline{S_0})$$

$$r_c = (\overline{S_2} \cdot \overline{S_1} \cdot S_0) \cdot \overline{p_c}$$

$$r_s = (\overline{S_2} \cdot S_1 \cdot \overline{S_0}) \cdot \overline{p_s}$$

$$r_e = (\overline{S_2} \cdot S_1 \cdot S_0) \cdot \overline{p_e}$$

$$r_v = (S_2 \cdot \overline{S_1} \cdot \overline{S_0}) \cdot \overline{p_v}$$

$$r_q = (D_1 \cdot p_c) + (D_2 \cdot p_s) + (D_3 \cdot p_e) + (D_4 \cdot p_v)$$

```
`timescale 1ns / 1ps

module ParkingController(
    input logic sys_clk,
    input logic sys_rst,
    input logic req_compact,
    input logic req_suv,
    input logic req_elec,
    input logic req_vip,

    input logic is_full_c,
    input logic is_full_s,
    input logic is_full_e,
    input logic is_full_v,

    output logic grant_c,
    output logic grant_s,
```

```
output logic grant_e,
output logic grant_v,
output logic en_queue
);

typedef enum logic [2:0] {
    ST_WAIT      = 3'b000,
    ST_CHK_CMP   = 3'b001,
    ST_CHK_SUV   = 3'b010,
    ST_CHK_ELEC  = 3'b011,
    ST_CHK_VIP   = 3'b100
} fsm_state_t;

fsm_state_t state_curr, state_next;


always_ff @(posedge sys_clk or posedge sys_rst) begin
    if (sys_rst) begin
        state_curr <= ST_WAIT;
    end else begin
        state_curr <= state_next;
    end
end


always_comb begin
    state_next = state_curr;

    if (state_curr == ST_WAIT) begin
```

```
        if (req_compact)       state_next = ST_CHK_CMP;
        else if (req_suv)      state_next = ST_CHK_SUV;
        else if (req_elec)     state_next = ST_CHK_ELEC;
        else if (req_vip)      state_next = ST_CHK_VIP;
        else                   state_next = ST_WAIT;
    end
    else begin
        state_next = ST_WAIT;
    end
end

always_comb begin
    grant_c = 0; grant_s = 0; grant_e = 0; grant_v = 0;
    en_queue = 0;

    case (state_curr)
        ST_CHK_CMP: begin
            if (~is_full_c) grant_c = 1;
            else            en_queue = 1;
        end
        ST_CHK_SUV: begin
            if (~is_full_s) grant_s = 1;
            else            en_queue = 1;
        end
        ST_CHK_ELEC: begin
            if (~is_full_e) grant_e = 1;
            else            en_queue = 1;
        end
```

```
            ST_CHK_VIP: begin
                if (~is_full_v) grant_v = 1;
                else            en_queue = 1;
            end
            default: begin

`timescale 1ns / 1ps

module tb_ParkingSystem;
    logic clk, reset;
    logic btn_c, btn_s, btn_e, btn_v;
    logic full_c, full_s, full_e, full_v;
    logic out_c, out_s, out_e, out_v;
    logic out_queue;

    localparam CLK_PERIOD = 10;
    ParkingController dut (
        .sys_clk(clk),
        .sys_rst(reset),
        .req_compact(btn_c), .req_suv(btn_s), .req_elec(btn_e), .req_vip(btn_v),
        .is_full_c(full_c), .is_full_s(full_s), .is_full_e(full_e), .is_full_v(full_v),
        .grant_c(out_c), .grant_s(out_s), .grant_e(out_e), .grant_v(out_v),
        .en_queue(out_queue)
    );


    initial begin
        clk = 0;
        forever #(CLK_PERIOD/2) clk = ~clk;
```

```
end

initial begin
    $display("Starting Simulation: Parking System Controller");

    reset = 1;
    btn_c = 0; btn_s = 0; btn_e = 0; btn_v = 0;
    full_c = 0; full_s = 0; full_e = 0; full_v = 0;
    #(CLK_PERIOD * 2);
    reset = 0;
    #(CLK_PERIOD);
    $display("Reset complete.");

    btn_c = 1;
    #(CLK_PERIOD);
    btn_c = 0; // Release button
    #(CLK_PERIOD/2);

    if (out_c === 1 && out_queue === 0)
        $display("[PASS] Scenario 1: Compact granted entry.");
    else
        $error("[FAIL] Scenario 1: Expected Grant Compact, got something else.");

    #(CLK_PERIOD); // Return to IDLE
    full_s = 1; // Mark SUV zone as full
    #(CLK_PERIOD);

    btn_s = 1;
    #(CLK_PERIOD);
```

```
btn_s = 0;

#(CLK_PERIOD/2);
if (out_s === 0 && out_queue === 1)
   $display("[PASS] Scenario 2: SUV sent to queue.");
else
   $error("[FAIL] Scenario 2: Expected Queue, got Grant or Nothing.");

#(CLK_PERIOD);

full_s = 0;
#(CLK_PERIOD);
btn_c = 1;
btn_s = 1;
#(CLK_PERIOD);
btn_c = 0;
btn_s = 0;

#(CLK_PERIOD/2);
if (out_c === 1 && out_s === 0)
   $display("[PASS] Scenario 3: Priority logic correct (Compact > SUV).");
else
   $error("[FAIL] Scenario 3: Priority logic failed.");
#(CLK_PERIOD * 2);
if (out_c == 0 && out_queue == 0)
   $display("[PASS] System returned to Idle state.");
else
   $error("[FAIL] System stuck in active state.");
```

```
        $display("Simulation Finished.");
        $stop;
    end
endmodule
```